RAG Assignment Report: Baseline vs. Re-Ranking

Name: Ishaan Giri **Roll no.:** 2025201091 **Repo:** github.com/Ishaan-Giri/lma-rag-reranking

1. Architecture Comparison

A bi-encoder passes the query and each chunk through the transformer *separately*, mean-pools each into one 384-d vector, and compares the two vectors with cosine similarity. The query and the chunk never see each other while being encoded. A cross-encoder instead takes `[CLS] query [SEP] chunk` as a single input, so self-attention runs across the tokens of both texts together and the model outputs one relevance score. This joint attention lets a query word attend directly to specific words in the chunk, which a single pooled vector cannot do.

Chunk vectors can be computed once, ahead of time, and stored in the index. The cross-encoder cannot be precomputed, because its score depends on the (query, chunk) pair, so every pair needs a full forward pass at query time.

Example from the rulebook. The Space Combat chunk (132) is mostly about combat steps, and the clause "if the defender announces a retreat, the attacker cannot announce a retreat" is only one bullet in it. A pooled vector is likely dominated by the general combat content, and the baseline ranked this chunk #7 for a query about the attacker retreating. The cross-encoder can match "attacker" and "retreat" to that exact bullet and ranked it #1.

2. Qualitative & Rank Shift Analysis

2.1 Rank changes for the failure queries

Out of 77 stress-test queries, re-ranking moved the right chunk from rank 4–9 into the top 3 in 10 cases.

Category	Query (short)	Right chunk	Rank change
Keyword trap	Can we take a trade back once it's done?	170 (Transactions)	#6 → #1
Keyword trap	Can I give commodities away for free?	170	#6 → #1
Keyword trap	Can I swap action cards with my neighbor?	170	#7 → #1
Keyword trap	Is there a limit on trading with one neighbor?	170	#6 → #1
Exception	Can I trade in the middle of a space combat?	170	#9 → #1
Exception	Any case where I can't score an objective I qualify for?	91 (Objectives)	#5 → #3
Negation	Can the attacker announce a retreat after the defender did?	132 (Space Combat)	#7 → #1
Negation	Can Sustain Damage save a ship from direct destruction?	153 (Sustain Damage)	#4 → #1
Buried	Do the Creuss wormholes stay if that player is eliminated?	58 (Elimination)	#4 → #1
Buried	Can I use a captured planet's resources right away?	53 (Control)	#6 → #2
Counter-example	Can I spend trade goods for extra votes?	23 / 79 / 161	#1 → #2

Overall, for all 77 queries: 56 were already correct in the baseline, **10 were fixed**, 5 were missed by both pipelines (in the top 10 but outside the top 3), and 6 were not in the top 10 at all, so re-ranking could not help.

Why the baseline failed. In the keyword-trap queries the baseline returned chunks about the *noun* in the question: "commodities" gave three Commodities chunks and "action cards" gave two

Action Cards chunks. They share words with the question but do not contain the answer, which sits in the Transactions rule (chunk 170) and is phrased in terms of the action (exchanging, giving, undoing).

Why chunk 170 appears five times. It is the one place where all the trading rules live, and topic chunks outrank it for every "trade" question. This also reflects how my stress set was built (several queries target the same rule), so these five are not five independent pieces of evidence.

Limits of a re-ranker. A re-ranker only reorders the candidates it is given. For example, the answer to KW02 ("when does the agenda phase start") was at bi-encoder rank #41, so no re-ranker could surface it; the fix there would be a larger top-k or a hybrid BM25 search. The re-ranker is also not perfect: it never pushed a baseline hit out of the top 3, but for some queries it moved the right chunk further down (e.g. KW07, #4 → #8), probably because the MS MARCO model is trained on web search and not on rulebooks.

2.2 Answer 1 (baseline top-3) vs. Answer 2 (re-ranked top-3)

Same prompt, same model, temperature 0. The prompt says "answer only from the excerpts; otherwise say they don't cover it."

Query	Answer 1	Answer 2	Gold in ctx. (A1/A2)
Take a trade back	Refused ("not covered")	Correct: cannot be undone	No / Yes
Give commodities free	Partial answer, then refused	Correct	No / Yes
Swap action cards	Refused	Correct	No / Yes
Trading limit	Refused	Correct	No / Yes
Trade during combat	Refused	Correct	No / Yes
Objective can't be scored	**Sounded complete but missed the home-system rule**	Correct, includes the home-system rule	No / Yes
Attacker retreat	Refused	Correct	No / Yes
Sustain Damage vs. destroy	Hedged, no clear answer	Correct: No	No / Yes
Creuss wormholes	Refused	Correct	No / Yes
Captured planet resources	Refused	Correct (needs two chunks)	No / Yes
Trade goods for votes	Correct (mentions Bribery card)	Correct (mentions Bribery card)	Yes / Yes

Hallucination, omission or misstatement? I saw no outright hallucination in either pipeline. The baseline failed by *omission*: on the objective query it listed "already scored" and "one per status phase" but missed the home-system rule, and on the commodities query it first explained the wrong rule (commodities turning into trade goods) before giving up. On the Sustain Damage query it only hedged.

Why refusals? 7 of the 10 baseline answers were refusals rather than wrong answers, because the prompt forces the model to answer only from the excerpts. I did not test it, but without that instruction Gemini would probably have answered from what it already knows about Twilight Imperium, and the retrieval failure would have been hidden.

Most dangerous failure. The objective query: the answer is fluent and looks complete, nothing in it signals that a rule is missing, and a player would wrongly believe they can score the objective. A refusal at least tells you something is wrong.

Counter-example. For the trade-goods query the re-ranker moved the Bribery errata to #1 and the gold chunk 23 fell out of the top 3, yet both answers were correct, because the same rule is

written in three places (chunks 23, 79, 161). Redundancy in the document protected the answer, so a rank change does not always change the final answer.

3. Latency Profiling

Measured on CPU (6 threads) after warm-up: 77 queries × 20 repetitions = 1540 samples per retrieval/re-rank stage. LLM time is from 22 real API calls.

Stage	Mean (ms)	Std (ms)	p95 (ms)
Retrieval: embed query + top-10 search	8.3	2.3	9.7
Re-rank: cross-encoder on 10 pairs	214.5	21.9	250.5
LLM generation (Gemini)	2819	1459	5672

Pipeline	Retrieval stage only	End-to-end (with LLM)
1: Baseline	8.3 ms	2828 ms
2: Re-ranking	222.8 ms	3042 ms

Retrieval becomes about 27× slower with re-ranking (8.3 → 222.8 ms), but the whole pipeline is only about 8% slower (2828 → 3042 ms). The numbers differ so much because the LLM call takes about 2.8 s in both pipelines, so the extra 0.2 s is small next to it (and smaller than the LLM's own variation, std 1.5 s). I think the cost is worth it here: for about 0.2 s, 10 of 77 queries went from a refusal or incomplete answer to a correct one. On a GPU the cross-encoder would be much faster (I did not measure this), so the cost would matter even less. For a real-time chatbot 200 ms is probably fine, but it would be noticeable if the target is the time to the first streamed token.

4. Scaling Question

Measured: cross-encoder on the top 10 ≈ 200 ms; on all 225 chunks ≈ 4.5 s (≈ 20 ms per pair); bi-encoder search ≈ 8 ms.

Collection size	Cross-encoder on everything (linear extrapolation)
225 chunks (this document)	4.5 s per query (measured)
10,000 chunks	≈ 201 s per query
1,000,000 chunks	≈ 20,100 s ≈ 5.6 hours per query

The cross-encoder needs one full transformer pass for every (query, chunk) pair, so for every single query its cost grows linearly with the number of chunks. None of this work can be cached or indexed in advance, because the input contains the query, which is not known until the user asks; there is no per-chunk representation that is independent of the query. The bi-encoder is different: chunks are embedded once offline (225 chunks took about 3.4 s), and each query then needs one encoder pass and a fast vector search (about 8 ms here).

The two-stage design uses each model for what it is good at: the bi-encoder gives *recall* cheaply by shortlisting 10 candidates out of everything, and the cross-encoder gives *precision* by carefully ordering only those 10, at a cost (about 200 ms) that does not depend on the collection size. The weakness is that the re-ranker can only reorder what the first stage found. In 6 of the 77 queries the right chunk was not in the top 10 (e.g. rank #41 for KW02), and the cross-encoder could do nothing for them. A larger top-k or a hybrid search (BM25 + dense) would help, but a larger top-k costs more cross-encoder time.